

NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

Prof Douglas Maskell

asdouglas@ntu.edu.sg

N4-02c-98

School of Computer Engineering

NTU

Course Site: https://ntulearn.ntu.edu.sg/

Schedule 2023

Week	Topic	Tutorial	Lecture 1 (Monday 8:30)	Lecture 2 (Wednesday 8:30)
1	Mod 1 (Intro, Review Verilog, Design Flow, IP)	No Tut	Introduction	Not used
2	Mod 2 (Verification & Verilog Testbenches)	Tut 1	Not Used	Not used
3	Mod 3 (FPGA Architecture and Synthesis)	Tut 2	Not used	Not Used
4	Mod 4 (Arithmetic on FPGAs)	Tut 3	Not used	**EC1** (Mod 1 & 2)
5	Mod 5 (Timing and Pipelining)	Tut 4	Not Used	**EC2** (Mod 3 & 4)
6	Mod 5 (Timing and Pipelining) continued	Tut 5	Not used	Not used
7	Mod 7 (**eLearning** Buses & interfacing)	No Tut	Not used	**QUIZ**
Recess				
8	Mod 8 (uP Case Study)	No Tut	Not Used	**EC3** (Mod 5 & 7)
9	Mod 8 (uP Case Study)	Tut 7	Not used	Not used
10	Mod 9 (Asynchronous Design)	Tut 8	Not used	**EC4** (Mod 8)
11	Mod 9 (Asynchronous Design)	Tut 9	Not Used	Not used
12	--	Tut 10	Not used	**EC5** (Mod 9)
13	--	Not used	Not used	Not used

The yellow shaded regions are for Example Classes (which are in addition to the LAMS-based lectures).
The green shaded region is for a class quiz (details will be provided later).
There is no Tutorial 6 as this has been integrated into the eLearning module.
The current Lab schedule can be accessed from: https://freeaccess.scse.ntu.edu.sg/scselabs

This may change slightly throughout the semester
Individual Tutors are likely to change the tutorial timing.



NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

Module 3

FPGA Architecture and Synthesis

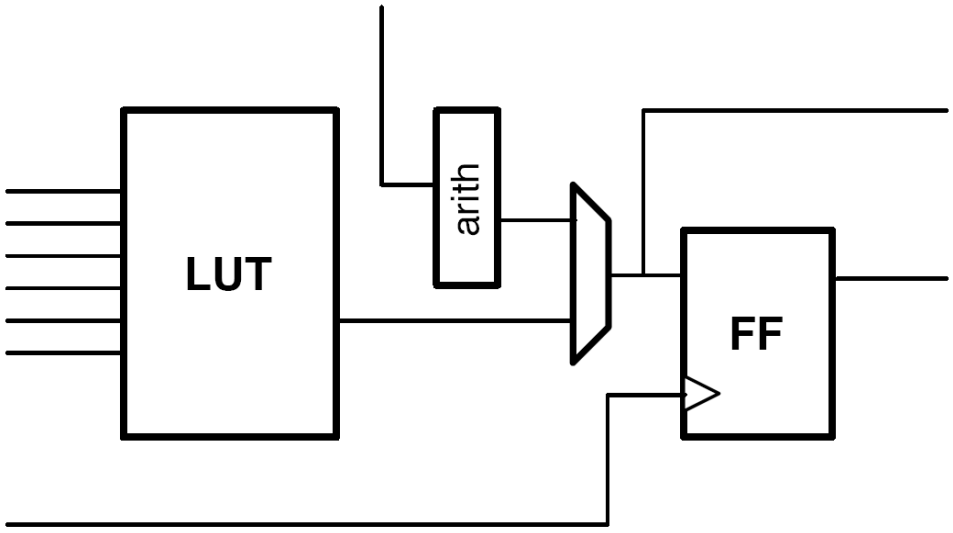
Summary

- Module 3: FPGA Architecture and Synthesis
  - Logic blocks: LUTs, FF, muxes and fast carry chain arithmetic
    - Formed into CLBs (2 Slices in a CLB, 4 LUTS in a Slice)
    - Can also be used as SRs, Distributed RAM and Multiplexers
  - Routing
  - Flexible I/O
  - Hard Macros: Block RAM, DSP, etc.
  - FPGA Synthesis: Some hints for efficient design

Logic: LUTs

The basic structure consists of:

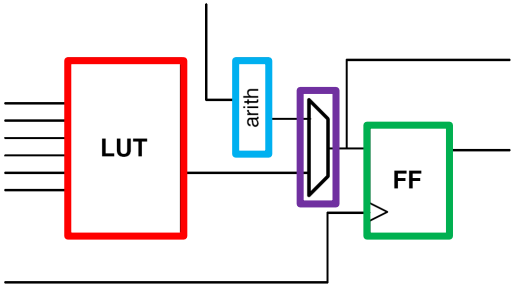
- A look-up table (LUT) to implement combinational logic functions
- Some auxiliary arithmetic logic
- Flip-flops that can be optionally enabled



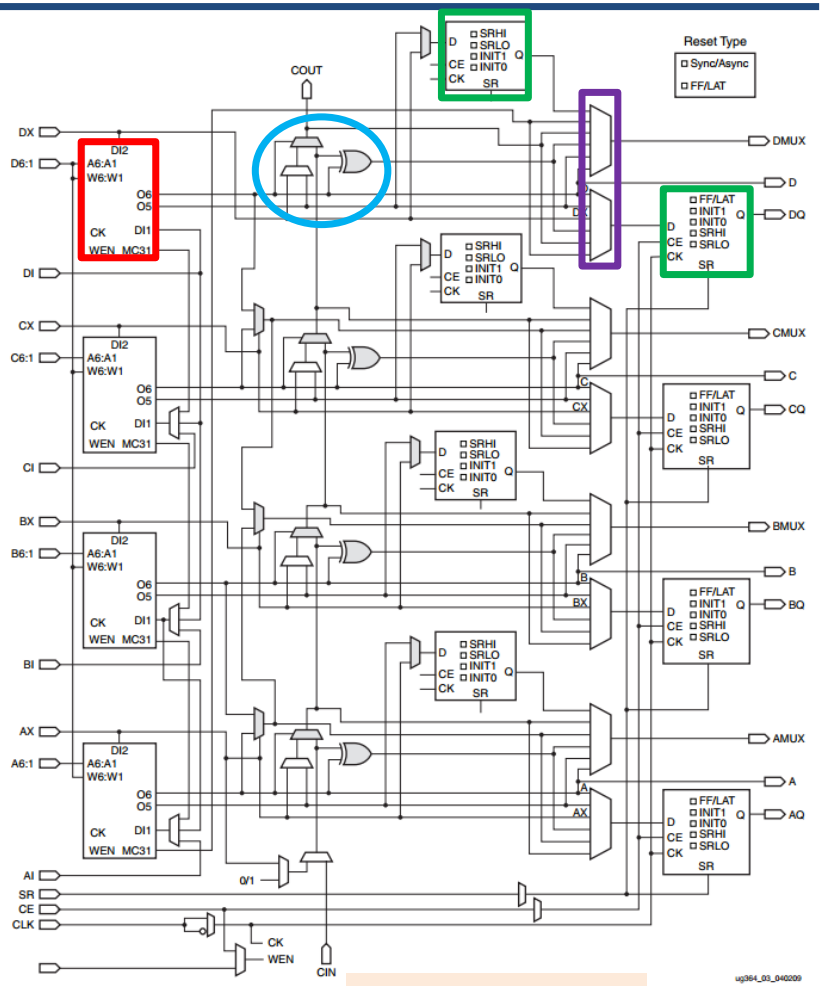
Logic: LUTs

The Xilinx slice structure is a little bit more complicated

A slice contains 4 of These



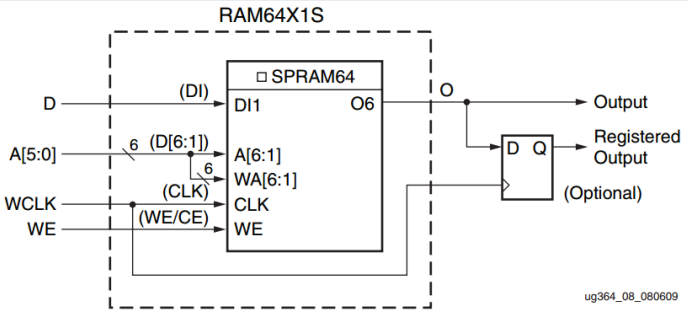
– There are 2 slices in a CLB



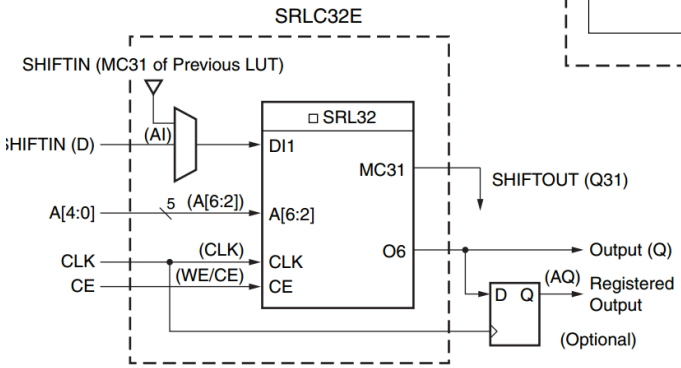
A Xilinx Slice

LUTs can also be used as”

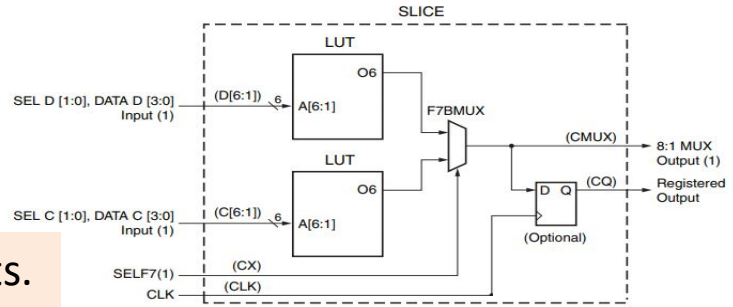
1. Distributed RAM (small memories)
   - Single/Dual/Quad port memories
   - Block RAM is better for large memories



2. Shift Registers
   - Efficient way to implement up to 32-bit SR
   - Can implement smaller SRs



1. Multiplexers
   - 1 LUT can implement a 4x1 mux
   - All 4 LUTs in a slice can give a 16x1 mux



How do we use them? Generally, the synthesiser will INFER these elements. The synthesiser will do this automatically as it will use less resource.
You can also instantiate these as primitives (e.g. SRL16E or SRLC32E). See https://docs.xilinx.com/r/en-US/ug974-vivado-ultrascale-libraries/Design-Elements

Routing and I/O

- Flexible routing matrix
  - Usually let the tools handle this
  - Has many wires, switch boxes & connection boxes

- Programmable I/O
  - Multi standard (Supports 3.3V, 2.5V, 1.8V, 1.5V, 1.35V 1.2V standards)
  - Single ended / differential
  - Best to just let the tools **infer** what you need
    - Use an .xdc constraints file to specify location, voltage standard, etc.

Hard Macros

- Modern FPGAs also have hard silicon components (Macros) in addition to the basic programable elements
  - Block Memories
    - Highly configurable: Ports, Depth, Width, Clocking (speed)
    - Can be configured as FIFOs
  - DSP blocks
    - Pre adder, Multiply, ALU
    - Dynamically configurable
    - But fixed widths. Multiplier is only 25x18-bit.
      - More than adequate for most DSP or Communications applications.
  - Also: Giga-bit Transceivers, ADC, Clocking, etc

How do we use them? Again, we would generally let the synthesiser INFER these elements.
For example, if you specify a multiplier, generally the synthesiser will automatically use a DSP block.
You can also use the IP Integrator (in Vivado) or directly instantiate the primitives (e.g. DSP48E2).
See: https://docs.xilinx.com/r/en-US/ug974-vivado-ultrascale-libraries/Design-Elements

Synthesis Techniques

- Synthesis optimisations, such as:
  - **Function reordering:** a complex function is reordered to reduce complexity
  - **Resource sharing:** where a complex resource can be shared without impacting the schedule
  - **Template matching:** If a set of instructions can be mapped directly to a hard block on the FPGA, this is done
  - **State reassignment:** The actual values used to represent FSM states are recomputed to optimise efficiency, typically a one-hot encoding

But this is not so important



NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

Module 4

Arithmetic on FPGAs

Summary

- Arithmetic Circuits
  - Ripple Carry Adders, Carry Look-ahead Adders, Multipliers
- Fixed-point Arithmetic
- Signed numbers in Verilog
- Floating point Numbers

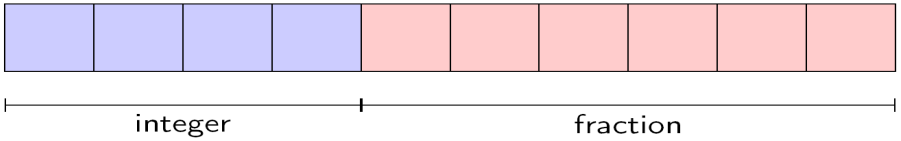
Number Formats

- Range of Integer Numbers (n-bit number)



  - Unsigned: 0 to $2^n - 1$ (0 to 1023 for n = 10)
  - Signed (2's Comp.): -2^{n-1} to $+2^{n-1} - 1$ (-512 to 511 for n = 10)

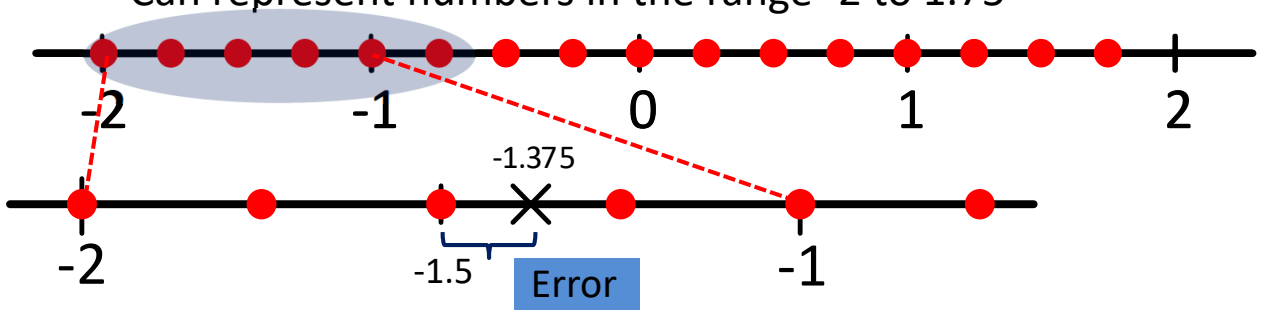
- Range of Fixed-point Numbers (p int. and q fract., $n=p+q$)



  - Unsigned: 0 to $2^p - 2^{-q}$ (0.0 to 15.984375 for p.q = 4.6)
  - Signed (2's Comp.): -2^{p-1} to $+2^{p-1} - 2^{-q}$ (-8.0 to 7.984375 for p.q = 4.6)

Fixed Point Arithmetic

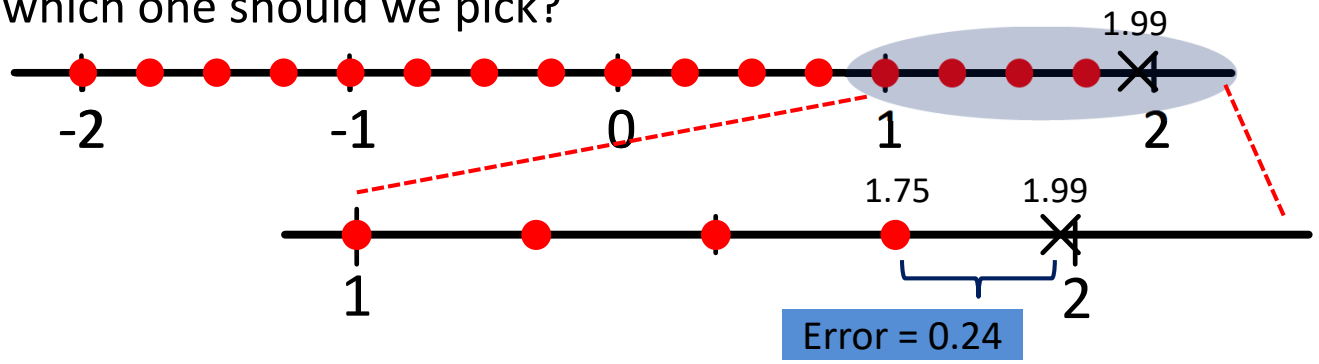
- Most fixed-point numbers **can not** be represented exactly. Introduces some error, depending on the format.
  - But FP numbers are also not exact and have some error.
- The max error is ½ of the least significant bit.
  - Consider numbers represented in 4 bits (p=q=2, eg 2.2 format)
    - Can represent numbers in the range -2 to 1.75



    - Consider the number -1.375. To convert, take magnitude, multiply by 2^{+q} (=2^2=4) and add 0.5. Then take integer part, add back sign and ÷ by 4
      - » 1.375 x 4 = 5.5 +0.5 = 6; -6 ÷ 4 = **-1.5**
    - Thus, the error is 0.125 (or ½ the least significant bit, or 2^{-3})

Fixed Point Arithmetic

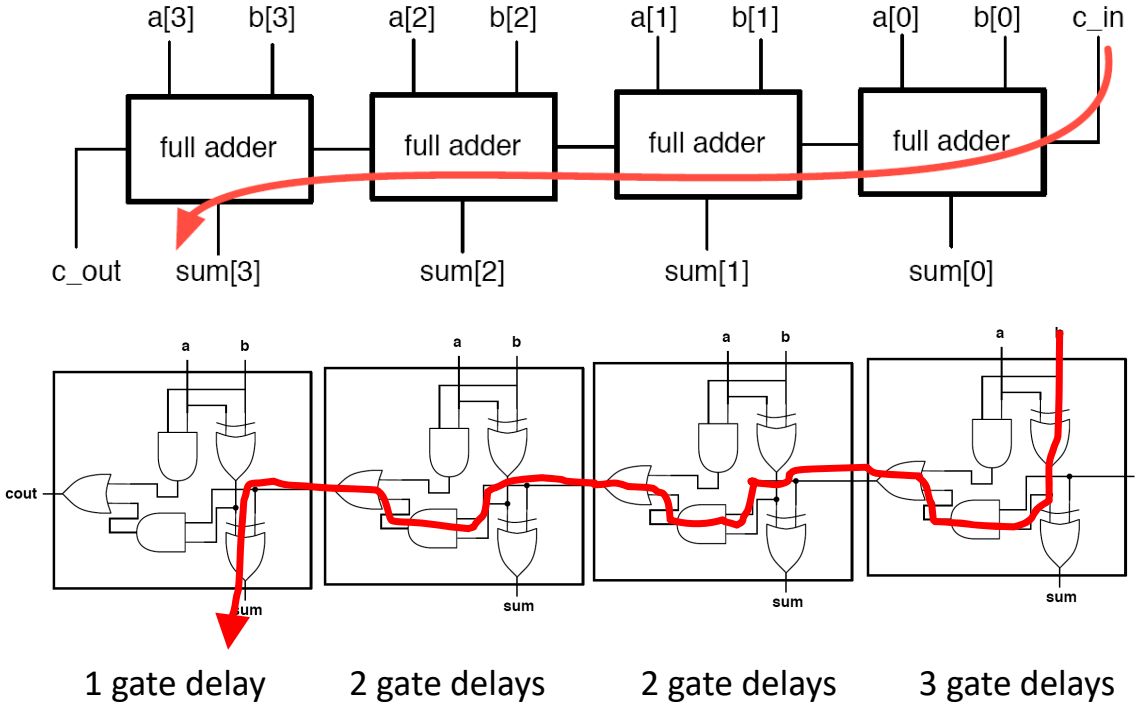
- What about the following:
  - Represent $-1.99 \leq x \leq 1.99$ as a 4-bit number with the smallest error
- Firstly, with 2 integer bits (p=q=2, eg 2.2 format), we can represent values in the range:
  - -2.0 to 1. 75. The worst-case error is 0.125. Note that 1.99 is outside this range
- Consider 3 integer bits (eg 3.1 format). Then we get a range:
  - -4.0 to 3.5. The worst-case error is 0.25. Now x is covered by the possible range
- So, which one should we pick?



- If we take the 2.2 format, the worst-case error is 0.24 (more than the 0.125 determined from 2^{-3}). But this is still less than the worst-case error of 0.25 if we choose the 3.1 format.
- So, we should use 2.2 format NOT 3.1 format.

adders

- Ripple adders
  - For large adders, the propagation delay through the adder is a problem



Total delay is 8 gate delays. Or 2n gate delays where n is the number of bits in the adder

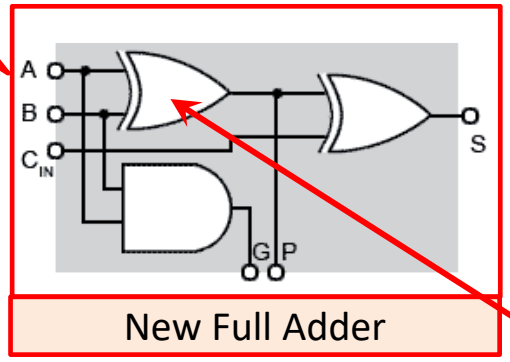
Adders: The need for speed

- Carry lookahead adders
  - Use generate (g) and propagate (p) signals
    - $g_i = a_i \,\&\, b_i$
    - $p_i = a_i \mid b_i$
  - $co_i = g_i \mid p_i \& ci_i$

a	b	cin	sum	cout	g	p
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	1	0	0	1
0	1	1	0	1	0	1
1	0	0	1	0	0	1
1	0	1	0	1	0	1
1	1	0	0	1	1	1
1	1	1	1	1	1	1

Adders: The need for speed

- Carry lookahead adders
  - Use generate (g) and propagate (p) signals
    - $g_i = a_i \& b_i$
    - $p_i = a_i \oplus b_i$
  - $co_i = g_i \mid p_i \& ci_i$



New Full Adder

a	b	cin	sum	cout	g	p
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	1	0	0	1
0	1	1	0	1	0	1
1	0	0	1	0	0	1
1	0	1	0	1	0	1
1	1	0	0	1	1	1
1	1	1	1	1	1	1

Note that we can change p to:
$p_i = a_i$ XOR b_i
as the last 2 terms are included in the g_i term. That way we can reuse the XOR from the Sum calculation

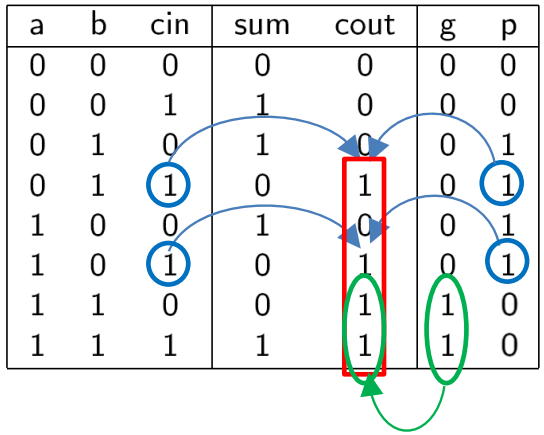
Can we simplify?

- Yes. We do not need the bottom term of p (the OR) as generate (g) will produce this
- So propagate (p) can become an XOR

Adders: The need for speed

- Carry lookahead adders
  - Use generate (g) and propagate (p) signals
    - $g_i = a_i \,\&\, b_i$
    - $p_i = a_i \oplus b_i$
  - $co_i = g_i \,|\, p_i \,\&\, ci_i$

a	b	cin	sum	cout	g	p
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	1	0	0	1
0	1	1	0	1	0	1
1	0	0	1	0	0	1
1	0	1	0	1	0	1
1	1	0	0	1	1	0
1	1	1	1	1	1	0



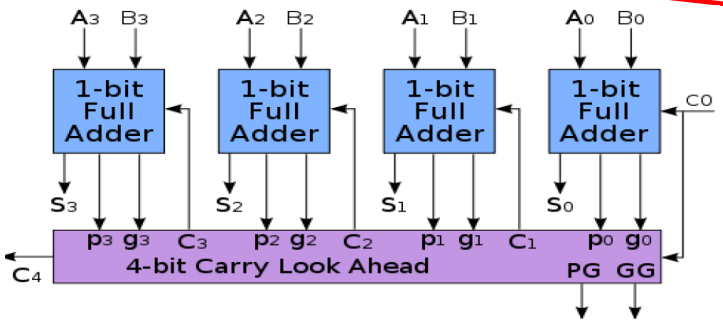
Which gives the same result as before

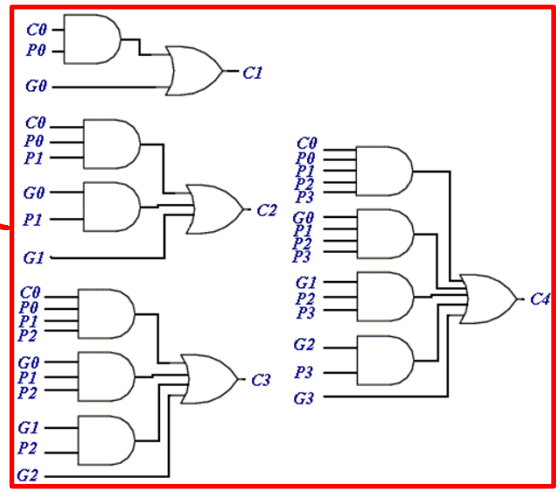
Adders: The need for speed

- Carry lookahead adders
  - Use generate (g) and propagate (p) signals
    - $g_i = a_i \,\&\, b_i$
    - $p_i = a_i \oplus b_i$
  - $co_i = g_i \mid p_i \& ci_i$
  - Thus, in a 4 stage CLA adder, the carry signals are derived as:
    - $c_1 = g_0 \mid p_0 \& c_0$
    - $c_2 = g_1 \mid p_1 \& c_1$

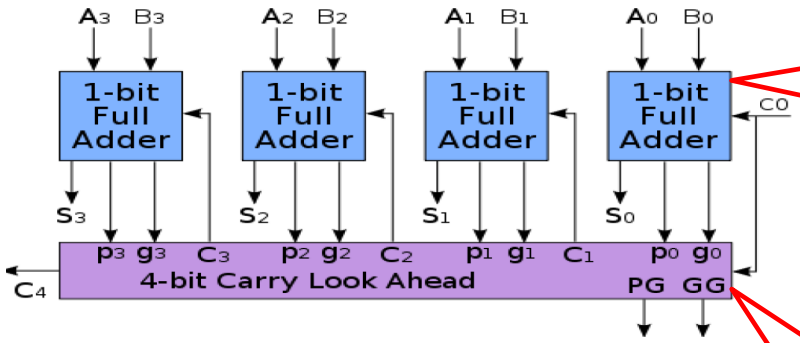
      $c_2 = g_1 \mid p_1 \& (g_0 \mid p_0 \& c_0) = g_1 \mid p_1 \& g_0 \mid p_1 \& p_0 \& c_0$, etc.

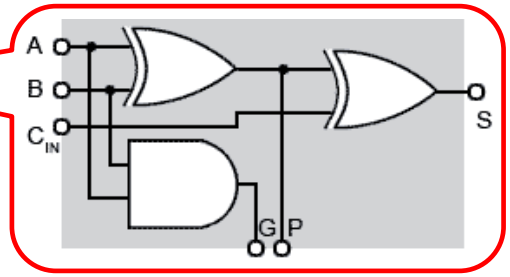
a	b	cin	sum	cout	g	p
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	1	0	0	1
0	1	1	0	1	0	1
1	0	0	1	0	0	1
1	0	1	0	1	0	1
1	1	0	0	1	1	1
1	1	1	1	1	1	1

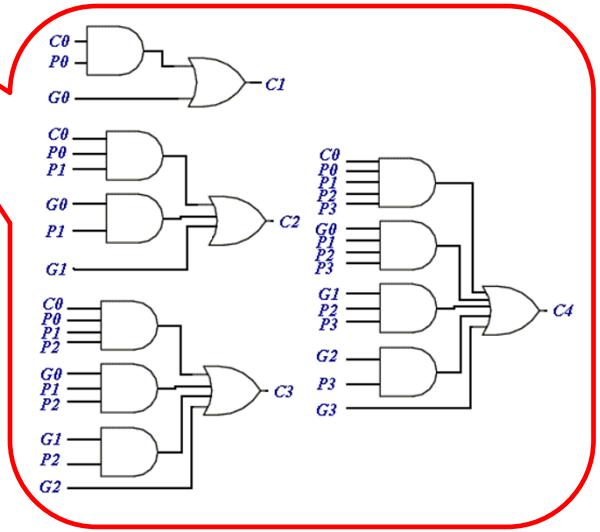




Delay through the CLA adder



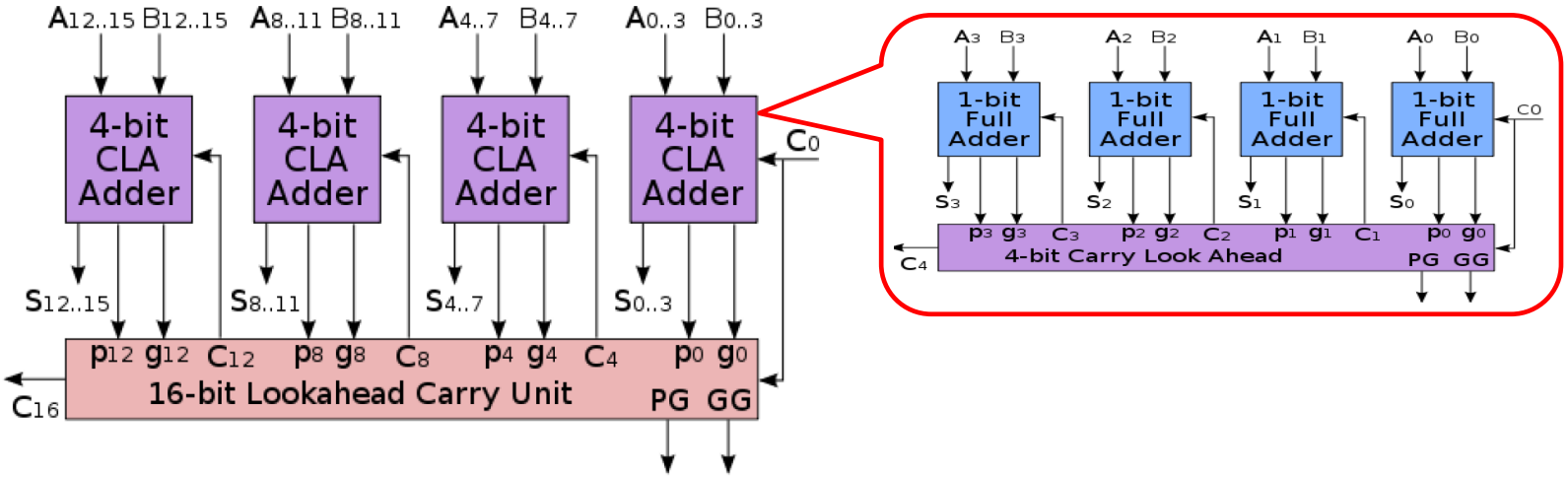




- The delay from A_0 or B_0 to C_4 is:
  - 1 in FA & 2 in CLA unit = **3 gate delays**
- The delay from A_0 or B_0 to S_3 is:
  - 1 in FA, 2 in CLA unit & 1 in last FA (FA_3)
    = 4 gate delays
  - **Compared to the 8 gate delays in the 4-bit ripple adder, this is about half.**

Note that this assumes that we can build the multi-input gates efficiently, and generally we can build gates with up to 4 or 5 inputs so that they are fast.

Larger CLA adders



- The 4-bit CLA unit is modified to add group generate (GG) & propagate (PG)

  Note: the group propagate (PG) and group generate (GG) for a 4-bit CLA are:

  $PG = p_0.p_1.p_2.p_3$ and $GG = g_3+g_2.p_3+g_1.p_3.p_2+g_0.p_3.p_2.p_1$

  – Results in no change to the delay (in the 4 bit CLA adder)
  – The 16-bit CLA unit is logically identical to the 4-bit CLA unit
    - Delay through the **16-bit CLA adder** is: 1 gate delays to generate the p's and g's, 2 gate delays to generate PG and GG, 2 gate delays to generate C_{12} & 3 gate delays for S_{15} (2 in the CLA unit and 1 in the FA of the LHS 4-bit CLA adder) **= 8 gate delays**.

But this complexity will NOT be examined.

Note can cascade further, combining 4 16-bit CLAs to make a 64-bit adder.
The total delay to generate the MSB of the Sum would be just **14 gate delays**.
This is considerably less than the **128 gate delays** for a 64-bit ripple carry adder.

Blank

Soln to Past Exams (CE2003)

18/19 Sem1 Q1(a): The Verilog always block describes a combinational function. Determine the contents, in hex, of a 4-input Look-Up Table (LUT) that implements this function. Assume an input order of a, b, c, d and output f.

```
always @ *
    begin
        f = 1'b1;
        if (a)
            if (b) f = c ^ d;
            else f = c & d;
        else if (c) f = ~a & d;
    end
```

Soln to Past Exams

18/19 Sem1 Q1(a): The Verilog always block describes a combinational function. Determine the contents, in hex, of a 4-input Look-Up Table (LUT) that implements this function. Assume an input order of a, b, c, d and output f.

```
always @ *
  begin
    f = 1'b1;
    if (a)
      if (b) f = c ^ d;
      else f = c & d;
    else if (c) f = ~a & d;
  end
```

LUT contents are (MSB to LSB):

= 0110_1000_1011_1011

= 68BB = 0x68BB

= 16'h68BB

a b c d	y	
0 0 0 0	1	LSB
0 0 0 1	1	
0 0 1 0	0	
0 0 1 1	1	
0 1 0 0	1	
0 1 0 1	1	
0 1 1 0	0	
0 1 1 1	1	
1 0 0 0	0	
1 0 0 1	0	
1 0 1 0	0	
1 0 1 1	1	
1 1 0 0	0	
1 1 0 1	1	
1 1 1 0	1	
1 1 1 1	0	MSB

Soln to previous Exams

- MCQ

(16) In the calculation of ***res* = *a*b* + *c*;**, a, b and c have bit lengths of 8, 10 and 12-bits, respectively. If $-0.8 \leq a \leq 0.8$, $-2.2 \leq b \leq 2.2$ and $-1.2 \leq c \leq 1.2$, determine the number of **fractional** bits in the 16-bit *res*, for the smallest worst-case error.

A. 10-bits
B. 11-bits
C. 12-bits
D. 13-bits
E. 14-bits

ANS:
a => 1.7; b => 3.7 and c => 2.10
so, $a*b$ has format 4.14, while ***a*b+c* has format 5.14**
However, as 16-bit *res* will have a numeric range ± 2.96
We only require 3 integer bits leaving 13 fractional bits

Soln to previous Exams

MCQ

(17) What are the contents of the 4-input LUT described by the following Verilog code? Assume an input order of *a*, *b*, *c*, *d* and output *f*.

```
f = 1'b0;
case ({a,b})
        2'b00:  f = c^d;
        2'b01:  f = ~c;
        2'b10:  f = c|d;
endcase
```

A. 16'h0E36
B. 16'h6C70
C. 16'hFA36
D. 16'h63FA
E. None of the above.

ANS:
0000_1110_0011_0110
Which is **16'h0E36**
(Remember MSB to LSB)

a	b	c	d	f
0	0	0	0	0
		0	1	1
		1	0	1
		1	1	0
0	1	0	0	1
		0	1	1
		1	0	0
		1	1	0
1	0	0	0	0
		0	1	1
		1	0	1
		1	1	1
1	1	0	0	0
		0	1	0
		1	0	0
		1	1	0

Soln to previous Exams

Q1 S2 17/18 (a) Digital designers describe circuits using hardware description languages (HDLs), like Verilog, or other tools that translate higher level descriptions into HDL. This is usually referred to as the *Design Entry* phase. The code is then processed through a sequence of steps. Write down the three steps in the FPGA design flow in order, and then briefly describe what happens in each step.

(9 marks)

- Synthesis: Verilog HDL is converted into a netlist of primitives. This is iterative with verification/simulation.
- Mapping: Netlists are mapped to the target technology's primitives. This step is usually included with synthesis in modern EDA tools.
- Placement: The primitives are placed to locations on the target device.
- Routing: The primitives (placed in the previous step) are connected as per the netlist. The router performs this as an iterative process trying to improve the routing results. After place and route, timing verification/simulation is carried out.

Now combined into the synthesis tool

Don't expect a question like this!

Soln to previous Exams

Q1 S2 17/18 (b) A carry-lookahead adder computes *generate* (g) and *propagate* (p) signals for each bit position from just the adder inputs (a and b). The g output indicates that a carry is produced regardless of the carry input. The p output indicates that a carry at the input will be propagated to the output. The equations for g and p at the ith bit position ($i = 0 \ldots N-1$) are:

$$g_i = a_i \,\&\, b_i;$$
$$p_i = a_i \mid b_i;$$

The carry out at the ith bit position (C_{i+1}) is determined by:

$$C_{i+1} = g_i \mid p_i \,\&\, C_i;$$

(i) Write the logic equation for the carry output at the 3rd stage (C_3) in terms of only the intermediate g and p signals and the carry into the 1st stage (C_0).

(6 marks)

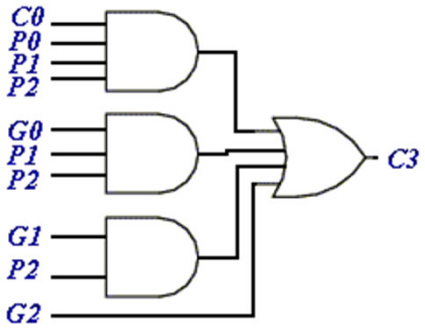
- $c_1 = g_0 | p_0 \& c_0$
- $c_2 = g_1 | p_1 \& c_1$
- $c_3 = g_2 | p_2 \& c_2$

$c_3 = g_2 | p_2 \& (g_1 | p_1 \& (g_0 | p_0 \& c_0)) = g_2 | p_2 \& g_1 | p_2 \& p_1 \& g_0 | p_2 \& p_1 \& p_0 \& c_0$

Soln to previous Exams

Q1 S2 17/18 (b) (ii) Based on your answer to Q1(b)(i), sketch the gate level implementation of C_3.

(5 marks)



(iii) Briefly explain why a carry-lookahead adder is often preferred to a standard ripple-carry adder in non-FPGA based hardware. Why isn't it applicable to FPGA?

(5 marks)

- In ripple-carry adders, the carry needs to ripple through all full adder modules before the final sum and carry can be generated. This is the critical path and impacts the adder speed.
- In a carry lookahead adder, the carry out from each of the lookahead modules can be calculated in just a few gate delays resulting in a reduction in the critical path.
- In FPGAs, the Slices have fast carry propagate logic which makes the LUT based adders fast without needing the complexity of an actual CLA implementation.

Soln to previous Exams

Q2 S2
17/18

The Verilog code shown below implements an arithmetic circuit, where *a*, *b* and *c* are 12-bit signed inputs, and *d* is a 24-bit signed input.

```
always @ (posedge clk) begin
    if (rst) begin
        a1 <= 0; b1 <= 0;
        c1 <= 0; d1 <= 0;
        res <= 0;
    end else begin
        a1 <= a; b1 <= b;
        c1 <= c; d1 <= d;
        res <= (a1 + b1) * c1 + d1;
    end
end
```

(d) If *a* and *b* have 3 integer and 9 fractional bits, and *c* has 5 integer and 7 fractional bits, determine the number of integer bits and fractional bits in the result (*res*), assuming *d* is suitably aligned. If we keep only 16 bits of the result, determined the exact range of numbers that can be represented.

(5 marks)

(e) What type of FPGA resource is ideally suited to implementing this circuit? Explain your answer.

(3 marks)

Soln to previous Exams

Q2 S2 17/18

The values a, b and c are 12-bit signed inputs, and d is a 24-bit signed input.

(d) If a and b have 3 integer and 9 fractional bits, and c has 5 integer and 7 fractional bits, determine the number of integer bits and fractional bits in the result (*res*), assuming d is suitably aligned. If we keep only 16 bits of the result, determined the exact range of numbers that can be represented.

ANS: a, b are 3.9 format while c is 5.7 format. Calculate $res = (a+b)*c + d$

$a+b$ is in 4.9 format (extra integer bit added for possible overflow); $(a+b)*c$ is 9.16

$(a+b)*c + d$ is 10.16. So 24-bit d will be in 9.15 format

***res* is thus 10.6 when converted to 16-bit**

So the range of signed numbers is: -2^9 to $+2^9 - 2^{-6}$ = -512 to 511.984375

(e) What type of FPGA resource is ideally suited to implementing this circuit? Explain your answer.

ANS: A small bit-width addition, followed by a multiplier, followed by a larger bit-width addition is ideal for implementing into a DSP block. The DSP block has a 25 bit preadder, 18x25 multiplier and a 48-bit adder/accumulator, so is OK.

(3 marks)

SC2103 Exam 2022 (MCQ)

(11) What are the contents of the 4-input LUT described by the following Verilog code? Assume input order of *a*, *b*, *c*, *d* and output *f*.

```
f = 1'b0;
case ({a,b})
    2'b00:    f = c^d;
    2'b10:    f = ~d;
    2'b11:    f = c&d;
endcase
```

A. 16'h60A1
B. 16'h6FA1
C. 16'h8306
D. 16'h8506
E. None of the above

a b	c d	f
0 0	0 0	0
	0 1	1
	1 0	1
	1 1	0
0 1	0 0	0
	0 1	0
	1 0	0
	1 1	0
1 0	0 0	1
	0 1	0
	1 0	1
	1 1	0
1 1	0 0	0
	0 1	0
	1 0	0
	1 1	1

Thus f = 0x8506

(14) In the calculation of ***res*** = ***a*b*** + ***c***;, *a*, *b* and *c* have bit lengths of 8, 10 and 12-bits, respectively. If $-0.99 \le a \le 0.99$, $-2.0 \le b \le 2.1$ and $-1.2 \le c \le 1.2$, determine the number of **fractional** bits in the 16-bit *res*, for the smallest worst case error.

A. 11-bits
B. 12-bits
C. 13-bits
D. 14-bits
E. None of the above

Thus ***a*** needs just 1 integer bit (the sign bit) -> **1.7**
b needs to be in **3.7** format
c needs to be in **2.10** format

So ***res*** will be **4.14** + **2.10** -> **5.14**
Worst case is 0.99*2.1 +1.2 = 3.279 -> 3 integer bits
So can round (in 16 bits) to **3.13** format

End of EC2